D

T A R E Q O

04 — Couche de données

La couche `lib/data/` implémente concrètement ce que déclare le domaine. Elle contient :

Version 2.1.3+23 · 14 septembre 2026

TAREQO

Table des matières

Structure 3

Deux implémentations, un contrat 3

Datasources distantes (Supabase) 4

- Agrégations côté serveur (W5/H2) 5

Persistance SQLite 6

- Où vit la base de données 6
- Schéma de la base de données 6
- Migrations légères (idempotentes) 8
- Aucun seed automatique 9

Unit of Work (transactions atomiques) 9

- Contrat (domaine) 9
- Implémentation SQLite (`SqliteUnitOfWork`) 9
- Transactions réentrantes (`AppDatabase`) 9
- Implémentation en mémoire 10

Repositories (implémentations) 10

Datasource distante (authentification) 10

Couche Comptes (multi-tenant) 10

Aucune donnée de démo 11

Documents connexes 11

- Datasources — sources de données locales (contrats + implémentations), métier distantes (Supabase) et auth distante ;
- Repositories — implémentations des contrats du domaine ;
- Unit of Work — transactions atomiques (SQLite) ;
- Catalogue de forfaits — la vraie grille de prix injectée à la composition root (sans données de démo).

Structure

```
lib/data/
 datasources/
  auth/
   auth_remote_data_source.dart # Contrat d'auth distante
   supabase_auth_remote_data_source.dart # Implémentation Supabase/gotrue
  remote/
   supabase_datasource_helpers.dart # Client + user_id de session
   supabase_product_data_source.dart # Produits (Supabase)
   supabase_sale_data_source.dart # Ventes (insert imbriqué sale + items)
   supabase_stock_lot_data_source.dart # Lots FIFO (Supabase)
   supabase_financial_transaction_data_source.dart # Transactions (Supabase)
   supabase_debt_data_source.dart # Dettes/crédit (Supabase)
   supabase_plan_data_source.dart # Forfaits (Supabase)
   supabase_tenant_data_source.dart # Tenants + approbation (Supabase)
   supabase_store_data_source.dart # Magasins (Supabase)
   supabase_employee_data_source.dart # Employés (Supabase)
   supabase_plan_request_data_source.dart # Évolutions (Supabase)
  local/
  product_local_data_source.dart # Contrat
  product_local_data_source_impl.dart # En mémoire
  product_sqlite_data_source_impl.dart # SQLite
  stock_lot_local_data_source.dart / _impl / _sqlite # Lots FIFO (contrat + mémoire + SQLite)
  sale_local_data_source.dart / _impl / _sqlite
  financial_transaction_local_data_source.dart / _impl / _sqlite
  cash_closing_local_data_source.dart / _impl / _sqlite # Clôture de caisse (Z-report)
  debt_local_data_source.dart / _impl / _sqlite
  plan_local_data_source.dart / _impl # Contrat + mémoire (Comptes)
  tenant_local_data_source.dart / _impl # Tenants (Comptes)
  store_local_data_source.dart / _impl # Magasins (Comptes)
  employee_local_data_source.dart / _impl # Employés (Comptes)
  plan_request_local_data_source.dart / _impl # Évolutions (Comptes)
  sqlite_unit_of_work.dart # IUnitOfWork réel (transaction)
  plan_catalog.dart # Forfaits (catalogue réel)
 repositories/
  product_repository_impl.dart
  sale_repository_impl.dart
  financial_transaction_repository_impl.dart
  debt_repository_impl.dart
  auth_repository_impl.dart
  plan_repository_impl.dart / tenant_repository_impl.dart
  store_repository_impl.dart / employee_repository_impl.dart
  plan_request_repository_impl.dart
```

Deux implémentations, un contrat

Chaque XxxLocalDataSource est une interface ; il y a deux implémentations :

Contrat	En mémoire (*_impl.dart)	SQLite (*_sqlite_impl.dart)
ProductLocalDataSource	List<Product> + seed	table products
StockLotLocalDataSource	List<StockLot> + seed	table stock_lots (ordre FIFO garanti dans la requête)
SaleLocalDataSource	List<Sale> + seed	tables sales + sale_items + sale_lot_consumption (une seule transaction)
FinancialTransactionLocalDataSource	List<FinancialTransaction> + seed	table financial_transactions
CashClosingLocalDataSource	List<CashClosing> + seed	table cash_closings (UPSERT par magasin + jour)
DebtLocalDataSource	List<Debt> + seed	table debts

Qui choisit laquelle ? Le ServiceLocator, par précédence :

```
if (SupabaseConfig.isConfigured) {
 _registerSupabaseDatasources(); // distant : SaaS, RLS par tenant
} else if (await AppDatabase.openAppDatabase()) {
 AppDatabase.registerDatasources(instance); // SQLite (natif)
} else {
 _registerInMemoryDatasources(); // en mémoire (web/repli)
}
```

> Avec SupabaseConfig.isConfigured = true (défaut, identifiants embarqués), les datasources distantes sont actives. Pour utiliser la persistance locale, lancez avec --dart-define=SUPABASE_URL= --dart-define=SUPABASE_ANON_KEY= (vide → isConfigured = false).

> Convention de parité : la liste en mémoire trie par nom sans tenir compte de la casse et SQLite utilise ORDER BY name COLLATE NOCASE — l'UI se comporte de la même façon avec les trois sources.

Datasources distantes (Supabase)

Elles implémentent les mêmes contrats (ProductLocalDataSource, SaleLocalDataSource, FinancialTransactionLocalDataSource, DebtLocalDataSource) avec le backend sur le Postgres de Supabase — le domaine, les cas d'usage et l'UI ne changent pas.

- Isolation tenant/magasin : chaque INSERT écrit user_id (depuis la session — requireCurrentUserId()) ; le RLS (auth.uid() = user_id OU public.is_my_store(store_id)) filtre dans la base, et les triggers set_tenant_user_id() + set_record_store_id() du schéma imposent les deux valeurs côté serveur (infalsifiable) ;
- Enums persistées par nom ; dates en TIMESTAMPTZ (ISO 8601) ; booléens en BOOLEAN ;

- Flux d'argent atomiques : les cas d'usage de vente/achat délèguent aux RPC `SECURITY DEFINER` (process_sale, process_purchase, register_debt, settle_debt) qui exécutent toute l'opération dans une transaction de base de données (vente + articles + lots de stock FIFO + caisse, ou dette + entrée en attente, ou règlement + résolution) avec clés d'idempotence et verrouillage de stock FOR UPDATE. process_sale consomme les lots en ordre FIFO ; process_purchase crée les lots de l'entrée (avec péremption facultative). L'insert imbriqué PostgREST (vente + articles en une requête) reste le add de repli du contrat ;
- Unit of Work : no-op en mode Supabase (chaque appel REST est sa propre transaction) — c'est pourquoi les RPC atomiques existent ;
- Sans session : les datasources lèvent StateError — les écrans de données ne sont atteignables qu'après connexion (AuthGate).

Agrégations côté serveur (W5/H2)

Pour passer à l'échelle, l'app ne télécharge pas tout l'historique pour additionner côté client : les agrégations lourdes s'exécutent dans Postgres comme RPC SECURITY DEFINER (garde is_my_store, valeurs BIGINT/centimes), et l'appareil ne reçoit que les chiffres — UN SEUL appel, payload constant. Le pattern couvre quatre fronts :

RPC	Repository	Agrège
get_dashboard_kpis (W5)	SupabaseDashboardKpisRepository	KPI du tableau de bord — chiffre d'affaires, ticket, compteurs — au lieu de sommer 4 listes côté client (GetDashboardKpisUseCase)
get_cash_flow (H2)	SupabaseCashFlowRepository	entrées/sorties payées de la période + solde (GetCashFlowUseCase)
get_sales_report (H2)	SupabaseSalesReportRepository	chiffre d'affaires, bénéfice brut, top-5 des ventes et ventes par jour dans le fuseau de l'appareil (GetSalesReportUseCase)
get_cash_closing_summary (H2)	SupabaseCashClosingSummaryRepository	aperçu Z-report : total/attendu dans le tiroir/répartition par moyen + clôture déjà enregistrée (GetCashClosingSummaryUseCase)

Comment fonctionne le câblage (DIP) : les cas d'usage reçoivent un serverRepository facultatif — enregistré uniquement en mode Supabase via instance.isRegistered<I...Repository>() dans le ServiceLocator (ex. ICashClosingSummaryRepository, ICashFlowRepository) ; local/tests reçoivent null et retombent sur le calcul côté client avec les mêmes critères que la RPC (ex. aggregateCashClosingDay reflète close_cash_register/get_cash_closing_summary). Le domaine et l'UI ne changent pas entre les modes — seul le câblage change, comme tout changement de persistance dans ce projet.

Lectures de listes (H2) : le complément des agrégations est la pagination par clé (fetchAllKeyset — curseur id stable, lots SupabaseQueryLimits.pageSize), qui a remplacé l'ancien .limit(1000) sur chaque getAll distant — des réponses petites et constantes même pour les magasins de 10k+ lignes.

> Voir le pattern complet, avec le diagramme du flux d'aperçu du Z-report, dans Architecture — Agrégations côté serveur (H2) ; les flux métier (clôture de caisse) dans Flux métier.

Le schéma (tables, RLS, politiques, index, triggers, fonctions et seed) vit dans `supabase/schema.sql`, validé par scripts/sql_check.sh contre PostgreSQL 16 (voir Tests).

Persistance SQLite

Où vit la base de données

- Fichier : `tareqo.db` dans getApplicationDocumentsDirectory() (via path_provider).
- PRAGMA foreign_keys = ON à l'ouverture.
- Sur le web SQLite ne s'exécute pas (dart:ffi n'existe pas) → import conditionnel vers app_database_stub.dart → en mémoire. Dans les tests sur la VM Windows, package:sqlite3 retombe sur la winsqlite3.dll système (voir test/helpers/test_sqlite.dart).

Schéma de la base de données

Depuis W4, l'argent est en centimes INTEGER ; depuis W11, les quantités sont en milli-unités INTEGER (1000 = 1 unité/kg) — même philosophie que les centimes, aucun flottant pour l'argent ni les quantités.

```
CREATE TABLE products (
 id TEXT PRIMARY KEY,
 store_id TEXT NOT NULL DEFAULT 'st1', -- Phase 2 : portée magasin
 name TEXT NOT NULL,
 description TEXT NOT NULL DEFAULT '',
 purchase_price INTEGER NOT NULL CHECK (purchase_price >= 0), -- centimes (W4)
 sale_price INTEGER NOT NULL CHECK (sale_price >= 0), -- centimes (W4)
 stock_quantity INTEGER NOT NULL DEFAULT 0 CHECK (stock_quantity >= 0), -- milli-unités (W11)
 code TEXT, -- code-barres d'étiquette/PLU (W11, facultatif)
 venda_por_fracao INTEGER NOT NULL DEFAULT 0, -- 0/1 : vendu au poids (W11)
 unit_label TEXT NOT NULL DEFAULT 'un' -- kg | un (W11)
);
CREATE TABLE sales (
 id TEXT PRIMARY KEY,
 store_id TEXT NOT NULL DEFAULT 'st1',
 sale_number INTEGER NOT NULL DEFAULT 0, -- séquentiel par magasin (S-00001)
 total_amount INTEGER NOT NULL CHECK (total_amount >= 0), -- centimes
 payment_method TEXT NOT NULL, -- nom de l'enum (stable)
 date_time INTEGER NOT NULL, -- millis epoch
 status TEXT NOT NULL DEFAULT 'active', -- active | cancelled (annulation)
 cancelled_at INTEGER -- moment de l'annulation (NULL tant qu'active)
);
CREATE TABLE sale_items (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 sale_id TEXT NOT NULL REFERENCES sales(id) ON DELETE CASCADE,
 product_id TEXT NOT NULL,
 product_name TEXT NOT NULL, -- snapshot
 quantity INTEGER NOT NULL CHECK (quantity > 0), -- milli-unités (W11)
 unit_price INTEGER NOT NULL CHECK (unit_price >= 0) -- centimes
);
CREATE INDEX idx_sale_items_sale_id ON sale_items(sale_id);
-- Consommation de LOTS par vente — écrite dans la MÊME transaction que la vente.
-- Elle permet à l'annulation de rendre le stock aux lots EXACTS consommés
-- (en préservant la péremption et le coût de chaque lot).
CREATE TABLE sale_lot_consumption (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 sale_id TEXT NOT NULL REFERENCES sales(id) ON DELETE CASCADE,
 lot_id TEXT NOT NULL,
 product_id TEXT NOT NULL,
```

```
 quantity INTEGER NOT NULL CHECK (quantity > 0) -- milli-unités (W11)
);
CREATE INDEX idx_sale_lot_consumption_sale ON sale_lot_consumption(sale_id);
CREATE TABLE financial_transactions (
 id TEXT PRIMARY KEY,
 store_id TEXT NOT NULL DEFAULT 'st1',
 description TEXT NOT NULL,
 amount INTEGER NOT NULL CHECK (amount >= 0), -- centimes
 type TEXT NOT NULL, -- income | expense
 category TEXT NOT NULL,
 date INTEGER NOT NULL, -- millis epoch
 status TEXT NOT NULL DEFAULT 'paid', -- paid | pending
 debt_id TEXT -- lien vers debts (crédit)
);
CREATE TABLE debts (
 id TEXT PRIMARY KEY,
 store_id TEXT NOT NULL DEFAULT 'st1',
 debt_number INTEGER NOT NULL DEFAULT 0, -- séquentiel par magasin (D-00001)
 person_name TEXT NOT NULL,
 amount INTEGER NOT NULL CHECK (amount >= 0), -- centimes
 due_date INTEGER NOT NULL, -- millis epoch
 is_customer_debt INTEGER NOT NULL, -- 0/1
 is_paid INTEGER NOT NULL DEFAULT 0, -- 0/1
 paid_at INTEGER, -- millis epoch (date du règlement)
 customer_id TEXT -- lien vers customers (module crédit)
);
CREATE TABLE customers (
 id TEXT PRIMARY KEY,
 store_id TEXT NOT NULL DEFAULT 'st1',
 name TEXT NOT NULL,
 phone TEXT NOT NULL DEFAULT '',
 email TEXT NOT NULL DEFAULT ''
);
CREATE TABLE purchases (
 id TEXT PRIMARY KEY,
 store_id TEXT NOT NULL DEFAULT 'st1',
 purchase_number INTEGER NOT NULL DEFAULT 0, -- séquentiel par magasin (C-00001)
 supplier_name TEXT NOT NULL,
 total_amount INTEGER NOT NULL CHECK (total_amount >= 0), -- centimes
 payment_method TEXT NOT NULL DEFAULT 'cash', -- moyen de paiement du fournisseur
 date_time INTEGER NOT NULL
);
CREATE TABLE purchase_items (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 purchase_id TEXT NOT NULL REFERENCES purchases(id) ON DELETE CASCADE,
 product_id TEXT NOT NULL,
 product_name TEXT NOT NULL,
 quantity INTEGER NOT NULL CHECK (quantity > 0), -- milli-unités (W11)
 unit_price INTEGER NOT NULL CHECK (unit_price >= 0), -- centimes
 expiry_date INTEGER, -- péremption du lot créé par cet article (W11)
 UNIQUE (purchase_id, product_id)
);
CREATE INDEX idx_purchase_items_purchase_id ON purchase_items(purchase_id);
-- W11 — lots FIFO : chaque entrée en stock crée un lot ; la diminution au PDV
-- consomme par péremption (la plus proche d'abord) et, sans péremption, par
-- date d'entrée.
CREATE TABLE stock_lots (
 id TEXT PRIMARY KEY,
 store_id TEXT NOT NULL DEFAULT 'st1',
 product_id TEXT NOT NULL,
 entry_date INTEGER NOT NULL, -- millis epoch (date d'entrée)
 expiry_date INTEGER, -- péremption (facultatif, NULL = aucune)
 initial_quantity INTEGER NOT NULL CHECK (initial_quantity >= 0), -- milli-unités
 current_quantity INTEGER NOT NULL CHECK (current_quantity >= 0), -- milli-unités
 cost_price INTEGER NOT NULL CHECK (cost_price >= 0) -- centimes
);
CREATE INDEX idx_stock_lots_product ON stock_lots(product_id, store_id);
CREATE INDEX idx_stock_lots_expiry ON stock_lots(expiry_date) WHERE current_quantity > 0;
-- Clôture de caisse du jour (Z-report) — une clôture par magasin par jour.
```

```
-- closing_date = début du jour local en ms (epoch) ; expected_cash = ce qui
-- doit être dans le tiroir (Σ des ventes ACTIVES en ESPÈCES du jour) ;
-- counted_cash = espèces physiques comptées lors de la conférence ;
-- sales_by_method = répartition par moyen (JSON).
CREATE TABLE cash_closings (
 id TEXT PRIMARY KEY,
 store_id TEXT NOT NULL DEFAULT 'st1',
 closing_date INTEGER NOT NULL, -- millis epoch (début du jour local)
 total_sales INTEGER NOT NULL CHECK (total_sales >= 0), -- centimes
 expected_cash INTEGER NOT NULL CHECK (expected_cash >= 0), -- centimes
 counted_cash INTEGER NOT NULL CHECK (counted_cash >= 0), -- centimes
 sales_by_method TEXT NOT NULL DEFAULT '{}', -- JSON (method.name → centimes)
 closed_at INTEGER NOT NULL -- millis epoch
);
CREATE UNIQUE INDEX uq_cash_closings_store_day ON cash_closings(store_id, closing_date);
CREATE INDEX idx_cash_closings_store ON cash_closings(store_id);
```

Conventions de sérialisation :

Type	Persisté comme
Enum	Nom (payment_method, type, status) — stable en sérialisation
DateTime	Millis epoch (int)
bool	0/1
Argent	INTEGER — centimes (W4)
Quantité	INTEGER — milli-unités (W11 : 1000 = 1 unité/kg)

Les contraintes CHECK (stock_quantity >= 0, amount >= 0, quantity > 0) sont la dernière ligne de défense de la base ; les règles métier restent dans le domaine.

Migrations légères (idempotentes)

AppDatabase.createSchema applique des ALTER TABLE pour les bases déjà existantes :

1. `debts.is_paid` — ajouté avec DEFAULT 0 (les anciennes dettes restent en attente) ;
2. `financial_transactions.debt_id` — ajouté NULL (les anciennes dettes seront réglées via une nouvelle entrée, sans doublon) ;
3. `debts.paid_at` — ajouté NULL (les dettes déjà réglées n'ont pas de date de règlement avant un nouveau règlement) ;
4. `store_id` sur les 6 tables métier (Phase 2) — ajouté 'st1' + index par magasin.
5. `sales.sale_number` — séquentiel par magasin (S-00001…), rétro-rempli par ordre de date ; unique (store_id, sale_number) ;
6. `purchases.purchase_number` — séquentiel par magasin (C-00001…), rétro-rempli par ordre de date ; unique (store_id, purchase_number) ;
7. `debts.debt_number` — séquentiel par magasin (D-00001…), rétro-rempli par ordre d'échéance ; unique (store_id, debt_number) ;
8. `purchases.payment_method` — moyen de paiement du fournisseur (même enum que les ventes), ajouté 'cash' (les achats hérités étaient des dépenses immédiates en espèces) ;
9. `sales.status` + `sales.cancelled_at` (annulation de vente) — status avec DEFAULT 'active' (toutes les ventes existantes sont actives — jamais annulées) et cancelled_at NULL. Garde idempotente : présence de la colonne status ;

10. W11 (FIFO + péremption + milli-unités) — les bases héritées reçoivent les colonnes code/venda_por_fracao/unit_label (produits) et expiry_date (purchase_items) ; toutes les quantités (products, sale_items, purchase_items) sont converties d'unités entières en milli-unités (×1000) ; et chaque produit avec du stock reçoit un lot initial (lot-mig-…, sans péremption, coût = prix d'achat) pour que l'invariant stock = Σ lots tienne dès le premier jour. Garde idempotente : présence de la colonne code (ajoutée uniquement dans cette migration) — la conversion ×1000 ne se réexécute jamais.

Les migrations de numérotation s'exécutent après la migration store_id (les rétro-remplissages référencent store_id). Toutes utilisent PRAGMA table_info pour vérifier la colonne d'abord — sûres à réexécuter (les nouvelles bases créent déjà les colonnes dans CREATE TABLE).

Aucun seed automatique

La base de données ne seed jamais de données de démo : AppDatabase.initialize crée seulement le schéma et migre l'argent en centimes (W4) et les quantités en milli-unités (W11) — produits, ventes, caisse, crédit, clients et achats démarrent vides et n'enregistrent que les données réelles créées par l'utilisateur.

Unit of Work (transactions atomiques)

Contrat (domaine)

```
abstract interface class IUnitOfWork {
 Future<T> run<T>(Future<T> Function() action);
}
```

Implémentation SQLite (SqliteUnitOfWork)

- Exécute action() dans `BEGIN IMMEDIATE` → `COMMIT`/`ROLLBACK`.
- BEGIN IMMEDIATE acquiert le verrou d'écriture au début — ferme la fenêtre TOCTOU entre la pré-validation du stock et l'écriture.
- File FIFO interne : sérialise les transactions concurrentes dans le même processus (la connexion n'accepte pas de transactions imbriquées provenant de flux différents).

Transactions réentrantes (AppDatabase)

SQLite n'accepte pas les transactions imbriquées. C'est pourquoi AppDatabase garde un compteur de profondeur (_transactionDepth) :

- appel externe → émet le vrai BEGIN IMMEDIATE (passe à 1) ;
- appels imbriqués (ex. : SaleSqliteDataSourceImpl.add dans ProcessSaleUseCase via runInTransaction) → incrémentent seulement le compteur et rejoignent la transaction externe ;
- commit/rollback ne sont émis que lorsque le compteur revient à 0.

> Attention : Invariant global : le compteur est statique et suppose une seule connexion (singleton Database) et un IUnitOfWork par processus — comme l'app l'enregistre. N'ouvrez pas plusieurs connexions transactionnelles concurrentes.

Implémentation en mémoire

_DirectUnitOfWork (dans service_locator.dart) est un no-op : l'exécution en mémoire est déjà séquentielle — mais sans garantie d'atomicité en cas d'échec partiel (les écritures en mémoire n'échouent pratiquement jamais).

Repositories (implémentations)

Ce sont des ports passifs : ils délèguent seulement au datasource injecté.

Implémentation	Contrat du domaine	Délègue à
ProductRepositoryImpl	IProductRepository	ProductLocalDataSource
SaleRepositoryImpl	ISaleRepository	SaleLocalDataSource
FinancialTransactionRepositoryImpl	IFinancialTransactionRepository	FinancialTransactionLocalDataSource (le filtre getByPeriod s'exécute en Dart)
DebtRepositoryImpl	IDebtRepository	DebtLocalDataSource
AuthRepositoryImpl	IAuthRepository	AuthRemoteDataSource

Aucun repository ne sait si les données viennent de la mémoire, de SQLite ou d'une API — changer de persistance signifie changer un enregistrement dans le ServiceLocator.

Datasource distante (authentification)

- AuthRemoteDataSource — contrat (signUp, signIn, signOut, authStateChanges) ;
- SupabaseAuthRemoteDataSourceImpl — utilise Supabase.instance.client.auth (gotrue) et convertit ici l'utilisateur Supabase `User` en `AppUser` du domaine (les couches au-dessus ne voient jamais gotrue).

Détail important : lors de signUp, si la confirmation d'e-mail est activée, response.user peut être null (compte créé, session pas encore) → renvoie un AppUser avec un id placeholder (fallbackEmail) ; la vraie session n'arrive qu'après confirmation.

Couche Comptes (multi-tenant)

Les datasources Comptes (forfaits, tenants, magasins, employés et évolutions) suivent le même pattern à double implémentation : contrat + en mémoire (*_impl.dart) dans la branche locale et datasource Supabase (remote/) dans la branche distante. En dev local, la composition root n'injecte que le vrai catalogue de forfaits ; tenant, magasins, employés et évolutions démarrent vides — le propriétaire passe par l'onboarding et l'approbation. Détails du flux dans Multi-tenant et Super Admin.

Aucune donnée de démo

L'app ne seed jamais de contenu d'exemple (ni localement, ni dans le schéma de production) : la seule donnée injectée est le vrai catalogue de forfaits (PlanCatalog, reflété dans la Partie 2 de schema.sql). Produits, ventes, transactions, crédit, clients et achats démarrent vides — les périodes (Aujourd'hui/7 jours/Mois) ne reflètent que des données réelles. Les bases existantes ayant reçu l'ancien seed peuvent être nettoyées avec supabase/cleanup_demo_data.sql.

Documents connexes

- Architecture — où la couche de données s'insère.
- Flux métier — les transactions en action (vente atomique).
- Multi-tenant et Super Admin — forfaits, tenants, magasins et employés dans la base.
- Guide du développeur — comment exécuter les tests SQLite.